

Lecture Notes: Data and Model Versioning in MLOps

AI / MLOps Course

1 Why Versioning Matters in Machine Learning

Traditional software versioning focuses on **code**. Machine learning systems depend on **three evolving assets**:

- Code
- Data
- Models

Unlike software-only systems:

- Small data changes can cause large behavioral changes
- Models are **stateful artifacts**, not just code
- Reproducing results requires knowing *exact inputs*

Therefore, modern MLOps requires **explicit versioning of data and models**.

2 The Versioning Stack in MLOps

A clean separation of concerns:

Asset	Tool	Purpose
Code	Git	Source control
Data	DVC	Dataset versioning
Models	MLflow	Experiment and model lifecycle tracking

Each tool solves a **different problem**. None replaces the others.

3 Data Versioning with DVC

3.1 Motivation

Git is not suitable for data because:

- Datasets are large
- Datasets change frequently

- Git diffs are meaningless for binary data

DVC (Data Version Control) provides:

- Lightweight dataset versioning
- Reproducible pipelines
- Separation of code and data storage

3.2 Core Concepts in DVC

Tracked Data

Data files or directories managed by DVC instead of Git.

Pointer Files (.dvc)

Small text files tracked by Git that reference:

- File path
- Content hash
- Storage location

Cache

A content-addressed store where DVC keeps data objects by hash.

Stages

Pipeline steps defined by:

- Dependencies (inputs)
- Outputs
- Command

Stages form a Directed Acyclic Graph (DAG).

Remotes

External storage locations (local, S3, GCS, SSH, etc.) where data objects live.

3.3 What DVC Versions

- Raw datasets
- Processed datasets
- Pipeline outputs
- Reports and metrics

3.4 DVC Capabilities

- Content-hash-based data versioning
- Reproducible pipelines via `dvc.yaml`
- Incremental execution with `dvc repro`
- Data rollback via `Git commit + dvc checkout`

3.5 Best Practices for DVC

- Never commit raw data to Git
- Always commit `.dvc`, `dvc.yaml`, and `dvc.lock`
- Use `params.yaml` for tunable pipeline parameters
- Use remotes for collaboration

3.6 Common DVC Scenarios

- Dataset schema changes
- Reproducing an old experiment
- Comparing pipeline outputs across versions
- Sharing datasets across teams

4 Model Versioning with MLflow

4.1 Motivation

Saving model files (e.g., `.pkl`) is insufficient because:

- No record of training conditions
- No comparison across experiments
- No lifecycle management

MLflow treats models as **first-class versioned artifacts**.

4.2 What a Model Version Includes

A proper model version consists of:

- Model artifact (weights)
- Hyperparameters
- Training metrics

- Input/output signature
- Dataset reference
- Timestamp and author

4.3 Core Concepts in MLflow

Run

A single execution of training code.

Experiment

A collection of related runs.

Parameters

Configuration values used during training.

Metrics

Quantitative performance measures (accuracy, loss, etc.).

Artifacts

Files produced by a run (models, plots, reports).

Model Registry

A centralized store of named models with versions and stages.

4.4 Model Registry and Lifecycle

Each registered model can have multiple versions.

Stages:

- None (new)
- Staging (candidate)
- Production (active)
- Archived (retired)

Stage transitions are **metadata operations**, not retraining.

4.5 MLflow Capabilities

- Experiment comparison
- Automatic metric visualization
- Model versioning
- Deployment-friendly model loading
- Integration with most ML frameworks

4.6 Best Practices for MLflow

- Log parameters, metrics, and artifacts for every run
- Always log an input example and signature
- Never overwrite models; create new versions

4.7 Common MLflow Scenarios

- Hyperparameter tuning
- Model comparison
- Selecting best-performing model
- Promoting models to production

5 DVC vs MLflow: Clear Separation of Roles

Aspect	DVC	MLflow
Primary focus	Data	Models
Version unit	Content hash	Experiment run

6 Using DVC and MLflow Together

A mature MLOps workflow:

1. Use DVC to version datasets and pipelines
2. Train models using DVC-tracked data
3. Log models and metrics with MLflow
4. Record dataset hash as an MLflow parameter

This creates full lineage:

Which data produced which model, and why it was chosen.

7 Key Takeaways

- Data and models evolve independently
- Git alone is insufficient for ML systems
- DVC ensures data reproducibility
- MLflow ensures model traceability and lifecycle control
- Together, they form the backbone of MLOps